

The Phase Mirror "Oracle" MCP server is a governance-focused Model Context Protocol (MCP) implementation designed to provide a "glass-box" audit layer for AI agents. It serves as the primary interface for agents (like GitHub Copilot) to interact with the Phase Mirror Dissonance Protocol, enforcing strict governance semantics through a standardized response envelope.<sup>123</sup>

#### Core Functionality: The Governance Envelope

The defining feature of the MCP server is its normalization layer, which wraps all tool outputs in a `MCPGovernanceEnvelope`. This structural boundary ensures that governance policies are enforced regardless of prompt engineering or tool misconfiguration.<sup>123</sup>

#### Tier Enforcement:

Authoritative (Tier 1): Can emit binding outcomes like decision: "block" but only in a cloud environment with active stores.<sup>2</sup>

Experimental (Tier 2): Restricted to advisory outcomes; the normalizer automatically downgrades block to warn and strips L0-specific error codes.<sup>23</sup>

#### Environment Awareness:

Cloud Mode: Uses production stores (e.g., DynamoDB for False Positives).

Local Mode: All tools are marked `degradedMode: true` and restricted to non-binding advisory status.<sup>23</sup>

#### Exposed Governance Tools

The server implements a registry of tools categorized by their governance authority:<sup>12</sup>

| Tool Name | Tier | Primary Function |
|-----------|------|------------------|
|-----------|------|------------------|

|                                 |               |                                                                                       |
|---------------------------------|---------------|---------------------------------------------------------------------------------------|
| <code>analyze_dissonance</code> | Authoritative | Runs Adaptive Constraint Enforcement (ACE) to detect policy violations. <sup>12</sup> |
|---------------------------------|---------------|---------------------------------------------------------------------------------------|

|                                     |               |                                                                                            |
|-------------------------------------|---------------|--------------------------------------------------------------------------------------------|
| <code>validate_l0_invariants</code> | Authoritative | Executes low-latency checks for schema integrity, permission bits, and drift. <sup>2</sup> |
|-------------------------------------|---------------|--------------------------------------------------------------------------------------------|

|                                   |               |                                                                                                     |
|-----------------------------------|---------------|-----------------------------------------------------------------------------------------------------|
| <code>check_adr_compliance</code> | Authoritative | Verifies that code and configuration adhere to binding Architectural Decision Records. <sup>2</sup> |
|-----------------------------------|---------------|-----------------------------------------------------------------------------------------------------|

|                             |              |                                                                                      |
|-----------------------------|--------------|--------------------------------------------------------------------------------------|
| <code>query_fp_store</code> | Experimental | Provides read-only analytics on False Positive (FP) patterns and rates. <sup>2</sup> |
|-----------------------------|--------------|--------------------------------------------------------------------------------------|

|                                         |              |                                                                                                |
|-----------------------------------------|--------------|------------------------------------------------------------------------------------------------|
| <code>check_consent_requirements</code> | Experimental | Verifies if the required organizational consent exists for a specific operation. <sup>23</sup> |
|-----------------------------------------|--------------|------------------------------------------------------------------------------------------------|

#### Architecture & Safety Primitives

Bridge to Rust Engine: The server acts as a TypeScript-based orchestrator that dispatches performance-critical tasks (like KKT condition checks or lattice audits) to a compiled Rust binary via a JSON FFI bridge.<sup>1</sup>

Fail-Closed Design: In alignment with ADR-005, the server prioritizes integrity over availability. If internal validation or a tool fails, the system defaults to a "fail-closed" state to prevent unauthorized bypasses.<sup>345</sup>

Contract Enforcement: A CI/CD workflow (`mcp-contract.yml`) ensures that the server's capabilities always match its declared policy manifest. Every authoritative tool must be backed by a corresponding ADR.<sup>2</sup>

#### Integration Context

The MCP server is intended for use in "Governance-as-Compilation" workflows, where it provides real-time remediation guidance to developers. It redacts all sensitive data at the tool

boundary using deterministic HMAC-style nonces to ensure privacy while maintaining auditability.<sup>367</sup>

Would you like me to detail the specific L0 Invariant checks, such as the drift-magnitude or nonce-freshness protocols, that this server enforces?